Arquitetura de Sistema de Leitura e Classificação de Conversas WhatsApp com Baileys

Autor: Manus AI — Arquitetura de Software Sênior

Contexto: Documento técnico para implementação de sistema local de ingestão, processamento e classificação de conversas WhatsApp via Baileys (Node.js), sem envio de mensagens.

Sumário

1. Arquitetura Geral do Sistema
2. Integração com Baileys
3. Estratégia de Ingestão de Dados
4. Persistência Local com SQLite
5. Sincronização Incremental
6. Classificação de Conversas
7. Pipeline Completo de Processamento
8. Scheduler e Automação
9. Dashboard Operacional
10. Boas Práticas e Riscos

---

1. Arquitetura Geral do Sistema

1.1 Visão de Módulos

O sistema é composto por seis camadas funcionais independentes, cada uma com responsabilidade única e interface bem definida. A separação em módulos não é apenas estética — ela permite trocar a origem das mensagens (hoje Baileys, amanhã API oficial) sem reescrever a lógica de negócio.

Módulo	Responsabilidade	Tecnologia sugerida
`whatsapp/`	Conexão, autenticação, captura de eventos Baileys	`@whiskeysockets/baileys`

`ingestão/`	Normalização, deduplicação e enfileiramento de dados brutos	Fila em memória ou `better-queue`
`banco/`	Persistência, queries, migrações	`better-sqlite3` + `drizzle-orm`
`classificação/`	Motor de regras + chamada à IA externa	Lógica local + fetch para API LLM
`scheduler/`	Orquestração de sync periódico e classificação em lote	`node-cron`
`dashboard/`	API REST + frontend de visualização e ações operacionais	Express + React ou Svelte

1.2 Fluxo Completo dos Dados

O caminho de uma mensagem desde o WhatsApp até o dashboard segue um pipeline linear e auditável:

Plain Text

```
[WhatsApp Web] ──Baileys WebSocket──▶ [whatsapp/listener]
                                              |
                                      emite evento interno
                                              |
                                       [ingestão/normalizer]
                                              |
                                      salva raw_payload + normaliza
                                              |
                                       [banco/repository]
                                              |
                                      upsert em messages + conversations
                                              |
                                       [classificação/engine]
                                              |
                                      regras locais → (se inconclusivo) → LLM API
                                              |
                                      salva classification + history
                                              |
                                       [dashboard/api]
```

```
                                        |
                                expõe via REST para o frontend
```

A comunicação entre módulos deve ser feita via **EventEmitter interno** (não HTTP), o que mantém a latência baixa e evita dependências de rede entre partes do mesmo processo. O único ponto de saída de rede é a chamada à API de IA externa.

1.3 Responsabilidades por Camada

Camada de Conexão (`whatsapp/`): É a única parte do sistema que conhece o protocolo Baileys. Ela traduz eventos brutos do WebSocket em eventos de domínio (`message.received` , `chat.updated` , `contact.updated`) e os emite para o barramento interno. Nenhuma outra camada importa Baileys diretamente — isso é crítico para desacoplamento futuro.

Camada de Ingestão (`ingestão/`): Recebe os eventos de domínio, valida o schema, normaliza campos (remove prefixos `@s.whatsapp.net` , formata timestamps para Unix ms, extrai texto de diferentes tipos de mensagem) e chama o repositório. É aqui que mora a lógica de idempotência.

Camada de Banco (`banco/`): Expõe apenas funções de repositório (`upsertMessage` , `upsertConversation` , `saveClassification`). Não contém lógica de negócio. Toda query é preparada (prepared statements) para segurança e performance.

Camada de Classificação (`classificação/`): Primeiro aplica regras determinísticas locais (sem custo, sem latência de rede). Se o resultado for inconclusivo, monta o prompt e chama a API LLM. Salva o resultado e o histórico de versões.

Scheduler (`scheduler/`): Orquestra o ciclo de vida: a cada N minutos, busca conversas não classificadas ou desatualizadas, chama a camada de classificação em lote, e registra o run na tabela `sync_runs` .

Dashboard (`dashboard/`): Consome apenas o banco via repositório. Não acessa Baileys diretamente. Expõe endpoints REST para o frontend e permite ações operacionais (override de classificação, marcar como encerrado).

---

2. Integração com Baileys

2.1 Autenticação e Persistência de Sessão

Baileys usa o protocolo WhatsApp Web, que exige autenticação via QR code na primeira conexão. Após o scan, as credenciais são salvas localmente e reutilizadas nas reconexões subsequentes. A estratégia de persistência correta é usar o `useMultiFileAuthState` , que salva cada chave criptográfica em um arquivo separado — isso é mais robusto que o `useSingleFileAuthState` porque evita corrupção total do estado em caso de falha de escrita.

TypeScript

```
// whatsapp/auth.ts
import makeWASocket, { useMultiFileAuthState, DisconnectReason } from '@whiskey
import { Boom } from '@hapi/boom';
import path from 'path';

const AUTH_DIR = path.resolve('./data/auth');

export async function createConnection(eventBus: EventEmitter) {
  const { state, saveCreds } = await useMultiFileAuthState(AUTH_DIR);

  const sock = makeWASocket({
    auth: state,
    printQRInTerminal: true,       // QR no terminal na primeira vez
    syncFullHistory: false,        // não baixar histórico completo (pesado)
    markOnlineOnConnect: false,    // não aparecer como online
    generateHighQualityLinkPreview: false,
  });

  // Salvar credenciais sempre que atualizadas
  sock.ev.on('creds.update', saveCreds);

  return sock;
}
```

A opção `syncFullHistory: false` é essencial. Com `true`, o Baileys tenta baixar todo o histórico de mensagens do servidor, o que pode levar minutos e aumentar drasticamente o risco de bloqueio por comportamento anômalo.

2.2 Reconexão Automática

O Baileys emite o evento `connection.update` com o campo `lastDisconnect` quando a conexão cai. A lógica de reconexão deve distinguir entre desconexões esperadas (logout intencional, código 401) e inesperadas (timeout, erro de rede), reconectando apenas no segundo caso:

TypeScript

```
sock.ev.on('connection.update', async (update) => {
  const { connection, lastDisconnect, qr } = update;

  if (qr) {
    // Exibir QR para re-autenticação se sessão expirou
    console.log('Escaneie o QR code para autenticar');
  }
```

```
  if (connection === 'close') {
    const statusCode = (lastDisconnect?.error as Boom)?.output?.statusCode;
    const shouldReconnect = statusCode !== DisconnectReason.loggedOut;

    if (shouldReconnect) {
      console.log('Reconectando em 5s...');
      setTimeout(() => createConnection(eventBus), 5000);
    } else {
      console.error('Sessão encerrada. Apague ./data/auth e reinicie.');
    }
  }

  if (connection === 'open') {
    eventBus.emit('whatsapp.connected');
  }
});
```

O backoff exponencial (5s → 10s → 20s → máx 60s) é preferível ao delay fixo para evitar sobrecarga em falhas prolongadas.

2.3 Eventos Relevantes para Captura

Baileys expõe dezenas de eventos. Para um sistema somente-leitura, apenas três são necessários:

Evento Baileys	Quando dispara	O que capturar
`messages.upsert`	Nova mensagem recebida ou histórico sincronizado	`messages[]` , `type` (`notify` = nova, `append` = histórico)
`chats.upsert` / `chats.update`	Chat criado ou metadados alterados	`id` , `name` , `unreadCount` , `lastMessageTime`
`contacts.upsert` / `contacts.update`	Contato adicionado ou nome alterado	`id` , `name` , `notify` (nome de exibição)

O evento `messages.upsert` com `type === 'notify'` representa mensagens em tempo real. O `type === 'append'` representa mensagens do histórico sendo carregadas — deve ser tratado de forma diferente (sem disparar notificações, por exemplo).

TypeScript

```
sock.ev.on('messages.upsert', ({ messages, type }) => {
  for (const msg of messages) {
    if (msg.key.fromMe) continue; // ignorar mensagens enviadas por você
```

```
    eventBus.emit('message.received', { raw: msg, type });
  }
});

sock.ev.on('chats.upsert', (chats) => {
  for (const chat of chats) {
    eventBus.emit('chat.updated', chat);
  }
});
```

2.4 Limitações e Riscos do Baileys

Baileys é uma biblioteca não oficial que faz engenharia reversa do protocolo WhatsApp Web. Os riscos são reais e devem ser gerenciados conscientemente:

Risco de bloqueio de conta: O WhatsApp monitora padrões de uso anômalos. Contas que conectam via Baileys com comportamento atípico (muitas reconexões rápidas, acesso a histórico massivo, múltiplas sessões simultâneas) podem ser banidas temporária ou permanentemente. Para minimizar: use `syncFullHistory: false`, implemente backoff nas reconexões, e nunca rode mais de uma instância Baileys para o mesmo número.

Instabilidade da biblioteca: O protocolo WhatsApp Web muda periodicamente sem aviso. Atualizações do WhatsApp podem quebrar o Baileys até que a comunidade publique um patch. Monitore a versão e mantenha um processo de atualização controlado.

Sem SLA ou suporte oficial: Não há garantia de funcionamento. Para uso em produção crítica, avalie a API oficial do WhatsApp Business (Meta) como alternativa de longo prazo.

Limitação de multi-device: Baileys suporta o protocolo multi-device do WhatsApp, mas a sessão é vinculada a um dispositivo virtual. Se o número já tiver muitos dispositivos vinculados, pode haver conflitos.

---

3. Estratégia de Ingestão de Dados

3.1 Captura de Mensagens Novas

A captura em tempo real é feita via eventos Baileys (seção 2.3). O ponto crítico é o **normalizer**: a estrutura de uma mensagem Baileys é um objeto profundamente aninhado com dezenas de campos opcionais, dependendo do tipo (texto, imagem, áudio, documento, reação, etc.). O normalizer extrai apenas o que é relevante para o sistema:

TypeScript

```
// ingestão/normalizer.ts
export function normalizeMessage(raw: proto.IWebMessageInfo): NormalizedMessage
  const id = raw.key.id!;
  const chatId = raw.key.remoteJid!;
  const fromMe = raw.key.fromMe ?? false;
  const timestamp = Number(raw.messageTimestamp) * 1000; // converter para ms

  // Extrair texto de diferentes tipos de mensagem
  const text =
    raw.message?.conversation ||
    raw.message?.extendedTextMessage?.text ||
    raw.message?.imageMessage?.caption ||
    raw.message?.videoMessage?.caption ||
    raw.message?.documentMessage?.caption ||
    null;

  const messageType = Object.keys(raw.message ?? {})[0] ?? 'unknown';

  return {
    id,
    chatId,
    fromMe,
    timestamp,
    text,
    messageType,
    rawPayload: JSON.stringify(raw), // guardar payload bruto para auditoria
  };
}
```

3.2 Histórico e Sincronização

O Baileys com `syncFullHistory: false` ainda sincroniza um histórico recente (geralmente os últimos 3 meses ou últimas 100 mensagens por chat, dependendo do servidor). Esse histórico chega via `messages.upsert` com `type === 'append'`.

Para sincronização periódica (garantir que nenhuma mensagem foi perdida durante downtime), o scheduler pode chamar `sock.fetchMessageHistory(chatId, cursor, count)` para buscar mensagens antigas de um chat específico. Isso deve ser feito de forma conservadora — no máximo uma vez por hora por chat, e apenas para chats com atividade recente.

3.3 Idempotência e Deduplicação

A deduplicação é garantida pelo campo `id` da mensagem (gerado pelo WhatsApp, globalmente único por chat). O repositório deve usar `INSERT OR IGNORE` (SQLite) ou `ON`

`CONFLICT DO NOTHING` para garantir que a mesma mensagem nunca seja inserida duas vezes, independentemente de quantas vezes o evento for disparado:

```SQL
INSERT INTO messages (id, chat_id, from_me, timestamp, text, message_type, raw_
VALUES (?, ?, ?, ?, ?, ?, ?)
ON CONFLICT(id) DO NOTHING;
```

Para conversas (`chats`), a idempotência é feita via `UPSERT` — a conversa pode ser atualizada (novo `last_message_at` , novo `unread_count`), mas nunca duplicada:

```SQL
INSERT INTO conversations (id, name, last_message_at, unread_count)
VALUES (?, ?, ?, ?)
ON CONFLICT(id) DO UPDATE SET
  name = excluded.name,
  last_message_at = excluded.last_message_at,
  unread_count = excluded.unread_count;
```

3.4 Normalização de Dados

Os IDs do WhatsApp têm o formato `5511999999999@s.whatsapp.net` para pessoas e `5511999999999-1234567890@g.us` para grupos. A normalização deve:

- Remover o sufixo `@s.whatsapp.net` e `@g.us` para armazenar apenas o número/ID limpo.
- Formatar números brasileiros: remover o `55` do prefixo para exibição, mas manter o número completo no banco.
- Converter todos os timestamps para Unix milliseconds (evitar strings de data que dependem de locale).
- Marcar grupos vs. contatos individuais pelo sufixo original.

---

4. Persistência Local com SQLite

4.1 Por que SQLite

Para um sistema local com volume moderado (até ~500k mensagens), SQLite é a escolha correta. Ele elimina a necessidade de um servidor de banco de dados separado, tem performance excelente para leitura, e o arquivo `.db` pode ser copiado facilmente para backup. A biblioteca `better-sqlite3` é preferível à `sqlite3` assíncrona porque operações

síncronas simplificam o código e são mais performáticas para workloads de leitura intensiva.

4.2 Schema Completo

SQL

```
-- Contatos normalizados
CREATE TABLE contacts (
  id          TEXT PRIMARY KEY,   -- número limpo: "5511999999999"
  name        TEXT,               -- nome salvo na agenda
  display_name TEXT,              -- nome de exibição do WhatsApp
  is_business INTEGER DEFAULT 0,
  created_at  INTEGER NOT NULL,
  updated_at  INTEGER NOT NULL
);

-- Conversas (chats)
CREATE TABLE conversations (
  id              TEXT PRIMARY KEY,  -- JID completo: "5511999999999@s.whatsapp
  contact_id      TEXT REFERENCES contacts(id),
  name            TEXT,
  is_group        INTEGER DEFAULT 0,
  last_message_at INTEGER,
  unread_count    INTEGER DEFAULT 0,
  is_archived     INTEGER DEFAULT 0,
  created_at      INTEGER NOT NULL,
  updated_at      INTEGER NOT NULL
);

-- Mensagens individuais
CREATE TABLE messages (
  id           TEXT PRIMARY KEY,    -- ID único gerado pelo WhatsApp
  chat_id      TEXT NOT NULL REFERENCES conversations(id),
  from_me      INTEGER NOT NULL,
  timestamp    INTEGER NOT NULL,    -- Unix ms
  text         TEXT,                -- null para mídia sem legenda
  message_type TEXT NOT NULL,       -- "conversation", "imageMessage", etc.
  raw_payload  TEXT NOT NULL,       -- JSON bruto para auditoria
  created_at   INTEGER NOT NULL
);

-- Classificações atuais (uma por conversa)
CREATE TABLE classifications (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  conversation_id TEXT NOT NULL REFERENCES conversations(id),
  status          TEXT NOT NULL,    -- "lead", "cliente_ativo", "encerrado", et
```

```
  intent          TEXT,             -- "compra", "suporte", "duvida", etc.
  sentiment       TEXT,             -- "positivo", "neutro", "negativo"
  priority        INTEGER DEFAULT 3, -- 1=alta, 2=média, 3=baixa
  summary         TEXT,             -- resumo gerado pela IA
  next_action     TEXT,             -- sugestão de próximo passo
  classified_by   TEXT NOT NULL,    -- "rules" | "llm" | "manual"
  model_version   TEXT,             -- versão do modelo LLM usado
  classified_at   INTEGER NOT NULL,
  UNIQUE(conversation_id)           -- uma classificação ativa por conversa
);

-- Histórico de classificações (auditoria de mudanças)
CREATE TABLE classification_history (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  conversation_id TEXT NOT NULL,
  status          TEXT NOT NULL,
  intent          TEXT,
  sentiment       TEXT,
  priority        INTEGER,
  summary         TEXT,
  next_action     TEXT,
  classified_by   TEXT NOT NULL,
  model_version   TEXT,
  classified_at   INTEGER NOT NULL
);

-- Registro de execuções do scheduler
CREATE TABLE sync_runs (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  run_type        TEXT NOT NULL,    -- "realtime" | "scheduled" | "manual"
  started_at      INTEGER NOT NULL,
  finished_at     INTEGER,
  conversations_processed INTEGER DEFAULT 0,
  messages_ingested       INTEGER DEFAULT 0,
  classifications_updated INTEGER DEFAULT 0,
  error           TEXT,
  status          TEXT DEFAULT 'running' -- "running" | "success" | "error"
);

-- Índices para performance
CREATE INDEX idx_messages_chat_id ON messages(chat_id);
CREATE INDEX idx_messages_timestamp ON messages(timestamp);
CREATE INDEX idx_classifications_status ON classifications(status);
CREATE INDEX idx_conversations_last_message ON conversations(last_message_at DE
```

4.3 Payload Bruto para Auditoria

A coluna `raw_payload TEXT` na tabela `messages` armazena o JSON completo do objeto Baileys. Isso serve para dois propósitos: (1) reprocessar mensagens com uma nova versão do normalizer sem precisar reconectar ao WhatsApp; (2) auditar o que exatamente foi recebido quando houver dúvidas sobre classificações. O custo de armazenamento é aceitável — uma mensagem de texto tem ~500 bytes de payload bruto.

---

5. Sincronização Incremental

5.1 Estratégia de Sync Eficiente

A sincronização periódica existe para cobrir dois cenários: mensagens perdidas durante downtime do sistema, e conversas que precisam ser reclassificadas porque acumularam novas mensagens desde a última classificação.

O sync incremental é baseado em dois campos: `last_message_at` da conversa e `classified_at` da classificação. A query para encontrar conversas que precisam de atenção é:

SQL

```
SELECT c.id, c.last_message_at, cl.classified_at
FROM conversations c
LEFT JOIN classifications cl ON cl.conversation_id = c.id
WHERE
  -- Nunca classificada
  cl.id IS NULL
  OR
  -- Classificação mais antiga que a última mensagem (nova atividade)
  cl.classified_at < c.last_message_at
ORDER BY c.last_message_at DESC
LIMIT 50; -- processar em lotes para não sobrecarregar a API LLM
```

5.2 Detectar Conversas Alteradas

Além do scheduler, o pipeline em tempo real deve marcar conversas como "dirty" (precisando reclassificação) sempre que uma nova mensagem chegar. A forma mais simples é atualizar `conversations.last_message_at` a cada mensagem recebida — o scheduler então detecta automaticamente que a classificação está desatualizada.

5.3 Evitar Reprocessamento Desnecessário

Para conversas muito ativas (dezenas de mensagens por hora), reclassificar a cada mensagem seria caro. A estratégia é usar um **debounce de classificação**: após receber uma mensagem, aguardar 5 minutos antes de classificar, cancelando o timer se uma nova

mensagem chegar nesse intervalo. Isso garante que a classificação seja feita com o contexto mais recente, não com uma mensagem isolada.

TypeScript

```
// scheduler/debounce.ts
const classificationTimers = new Map<string, NodeJS.Timeout>();

export function scheduleClassification(chatId: string, delay = 5 * 60 * 1000) {
  if (classificationTimers.has(chatId)) {
    clearTimeout(classificationTimers.get(chatId)!);
  }
  const timer = setTimeout(async () => {
    classificationTimers.delete(chatId);
    await classifyConversation(chatId);
  }, delay);
  classificationTimers.set(chatId, timer);
}
```

---

6. Classificação de Conversas

6.1 Motor de Regras Local

O motor de regras é a primeira linha de classificação. Ele é rápido (sem I/O), sem custo, e cobre os casos mais óbvios. As regras operam sobre o texto concatenado das últimas N mensagens da conversa:

TypeScript

```
// classificação/rules.ts
interface RuleResult {
  matched: boolean;
  status?: string;
  intent?: string;
  sentiment?: string;
  priority?: number;
  confidence: number; // 0-1
}

const RULES: Array<(text: string) => RuleResult> = [
  // Regra 1: Interesse explícito em compra
  (text) => {
    const buyKeywords = /\b(quero comprar|quanto custa|valor|preço|plano|assina
    if (buyKeywords.test(text)) {
      return { matched: true, status: 'lead_quente', intent: 'compra', priority
```

```
    }
    return { matched: false, confidence: 0 };
  },

  // Regra 2: Já é cliente (menciona nota fiscal, eSocial, etc.)
  (text) => {
    const clientKeywords = /\b(minha nota|meu esocial|meu informe|já uso|já ten
    if (clientKeywords.test(text)) {
      return { matched: true, status: 'cliente_ativo', intent: 'suporte', prior
    }
    return { matched: false, confidence: 0 };
  },

  // Regra 3: Conversa encerrada
  (text) => {
    const closedKeywords = /\b(obrigado|valeu|até mais|tchau|não tenho interess
    if (closedKeywords.test(text)) {
      return { matched: true, status: 'encerrado', intent: 'nenhum', priority:
    }
    return { matched: false, confidence: 0 };
  },
];

export function applyRules(conversationText: string): RuleResult {
  for (const rule of RULES) {
    const result = rule(conversationText);
    if (result.matched && result.confidence >= 0.75) {
      return result;
    }
  }
  return { matched: false, confidence: 0 };
}
```

6.2 Quando Chamar a IA Externa

A IA é chamada apenas quando as regras locais não produzem um resultado com confiança suficiente (threshold de 0.75). Isso reduz drasticamente o custo de API — na prática, regras simples cobrem 60-70% dos casos.

A decisão de chamar a IA também deve considerar: a conversa tem pelo menos 3 mensagens? O texto total tem pelo menos 50 caracteres? Conversas muito curtas ou sem texto (só mídia) não têm contexto suficiente para classificação significativa.

6.3 Estrutura do Resultado de Classificação

O resultado padronizado garante que o dashboard sempre receba o mesmo schema, independentemente de ter sido gerado por regras ou por IA:

TypeScript

```
interface ClassificationResult {
  status: 'lead_frio' | 'lead_quente' | 'cliente_ativo' | 'suporte' | 'encerrad
  intent: 'compra' | 'suporte' | 'duvida' | 'reclamacao' | 'nenhum' | null;
  sentiment: 'positivo' | 'neutro' | 'negativo';
  priority: 1 | 2 | 3; // 1=alta, 2=média, 3=baixa
  summary: string;       // 1-2 frases resumindo a conversa
  next_action: string;   // sugestão de próximo passo para o operador
  classified_by: 'rules' | 'llm' | 'manual';
  model_version?: string;
  confidence: number;    // 0-1
}
```

6.4 Prompt para o LLM

O prompt deve ser estruturado para produzir JSON diretamente, evitando parsing de texto livre:

TypeScript

```
// classificação/llm.ts
export async function classifyWithLLM(
  conversationText: string,
  contactName: string
): Promise<ClassificationResult> {
  const systemPrompt = `Você é um classificador de conversas de WhatsApp para u
Analise a conversa e retorne um JSON com os campos: status, intent, sentiment, 

Valores válidos:
- status: "lead_frio" | "lead_quente" | "cliente_ativo" | "suporte" | "encerrad
- intent: "compra" | "suporte" | "duvida" | "reclamacao" | "nenhum"
- sentiment: "positivo" | "neutro" | "negativo"
- priority: 1 (alta) | 2 (média) | 3 (baixa)
- summary: string (máximo 100 caracteres)
- next_action: string (máximo 80 caracteres)

Responda APENAS com o JSON, sem texto adicional.`;

  const userPrompt = `Contato: ${contactName}\n\nConversa:\n${conversationText.

  const response = await fetch(LLM_API_URL, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${LLM_API_KEY}`, 'Content-Type': 'appli
    body: JSON.stringify({
      messages: [
```

```
        { role: 'system', content: systemPrompt },
        { role: 'user', content: userPrompt },
      ],
      response_format: { type: 'json_object' },
    }),
  });

  const data = await response.json();
  return JSON.parse(data.choices[0].message.content);
}
```

6.5 Pipeline Híbrido para Redução de Custo

A estratégia de custo mínimo segue esta ordem de prioridade:

1. **Regras locais** (custo zero): cobrem ~65% dos casos com alta confiança.
2. **Cache de classificação recente**: se a conversa foi classificada há menos de 24h e não recebeu novas mensagens, reutilizar o resultado anterior.
3. **LLM apenas para casos ambíguos**: ~35% das conversas, com prompt otimizado para tokens mínimos (truncar texto em 3.000 caracteres).
4. **Classificação manual**: o operador pode fazer override via dashboard, que é salvo com `classified_by: 'manual'` e não é sobrescrito pelo scheduler automático.

---

7. Pipeline Completo de Processamento

7.1 Fluxo Detalhado

Plain Text

```
1. [Baileys] emite messages.upsert
       │
       ▼
2. [whatsapp/listener] filtra mensagens relevantes (não enviadas por mim, não d
   emite evento interno: message.received { raw, type }
       │
       ▼
3. [ingestão/normalizer] normaliza o objeto bruto → NormalizedMessage
       │
       ▼
4. [banco/repository.upsertMessage] INSERT OR IGNORE (idempotência)
   banco/repository.updateConversationLastMessage → atualiza last_message_at
       │
       ▼
```

```
5. [scheduler/debounce.scheduleClassification(chatId, 5min)]
   (aguarda 5 minutos para acumular mensagens antes de classificar)
       │
       ▼ (após debounce)
6. [classificação/engine.classifyConversation(chatId)]
   a) busca últimas 50 mensagens do chat no banco
   b) concatena textos
   c) applyRules(text) → se confidence >= 0.75, usar resultado
   d) senão, classifyWithLLM(text, contactName)
       │
       ▼
7. [banco/repository.saveClassification]
   INSERT OR REPLACE em classifications
   INSERT em classification_history (sempre, para auditoria)
       │
       ▼
8. [dashboard/api] expõe dados atualizados via GET /api/conversations
```

7.2 Versionamento de Classificações

A tabela `classification_history` registra cada mudança de classificação com timestamp e origem (`rules` , `llm` , `manual`). Isso permite:

- Auditar por que uma conversa foi classificada de determinada forma.
- Identificar conversas que mudaram de status muitas vezes (sinal de instabilidade ou conflito de regras).
- Treinar um modelo próprio no futuro usando o histórico de classificações manuais como ground truth.

TypeScript

```
// Ao salvar classificação, sempre inserir no histórico
async function saveClassification(result: ClassificationResult, chatId: string)
  const now = Date.now();

  // Atualizar classificação atual (UPSERT)
  db.prepare(`
    INSERT INTO classifications (conversation_id, status, intent, sentiment, pr
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    ON CONFLICT(conversation_id) DO UPDATE SET
      status = excluded.status, intent = excluded.intent,
      sentiment = excluded.sentiment, priority = excluded.priority,
      summary = excluded.summary, next_action = excluded.next_action,
      classified_by = excluded.classified_by, model_version = excluded.model_ve
      classified_at = excluded.classified_at
    WHERE classified_by != 'manual' -- não sobrescrever overrides manuais
```

```
  `).run(chatId, result.status, result.intent, result.sentiment, result.priorit
         result.summary, result.next_action, result.classified_by, result.model_

  // Sempre inserir no histórico
  db.prepare(`
    INSERT INTO classification_history (conversation_id, status, intent, sentim
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
  `).run(chatId, result.status, result.intent, result.sentiment, result.priorit
         result.summary, result.next_action, result.classified_by, result.model_
}
```

---

8. Scheduler e Automação

8.1 Estrutura do Scheduler

O scheduler usa `node-cron` para disparar dois tipos de jobs:

TypeScript

```
// scheduler/index.ts
import cron from 'node-cron';
import { runIncrementalSync } from './sync';
import { runBatchClassification } from './classification';

// Sync incremental a cada 30 minutos
cron.schedule('*/30 * * * *', async () => {
  await runWithLock('sync', runIncrementalSync);
});

// Classificação em lote a cada hora
cron.schedule('0 * * * *', async () => {
  await runWithLock('classification', runBatchClassification);
});
```

8.2 Evitar Execuções Concorrentes

O mecanismo de lock é crítico. Se o job de classificação demorar mais de 1 hora (por exemplo, por lentidão da API LLM), o próximo disparo do cron não deve iniciar uma segunda instância concorrente. O lock mais simples usa uma variável em memória:

TypeScript

```
// scheduler/lock.ts
const runningJobs = new Set<string>();
```

```
export async function runWithLock(jobName: string, fn: () => Promise<void>) {
  if (runningJobs.has(jobName)) {
    console.warn(`[Scheduler] Job "${jobName}" já está em execução. Pulando.`);
    return;
  }

  runningJobs.add(jobName);
  const runId = await db.prepare(
    `INSERT INTO sync_runs (run_type, started_at, status) VALUES (?, ?, 'runnin
  ).run(jobName, Date.now()).lastInsertRowid;

  try {
    await fn();
    db.prepare(`UPDATE sync_runs SET finished_at = ?, status = 'success' WHERE
      .run(Date.now(), runId);
  } catch (error) {
    db.prepare(`UPDATE sync_runs SET finished_at = ?, status = 'error', error =
      .run(Date.now(), String(error), runId);
    console.error(`[Scheduler] Erro no job "${jobName}":`, error);
  } finally {
    runningJobs.delete(jobName);
  }
}
```

Para sistemas com múltiplos processos (não é o caso aqui, mas pensando em escala), o lock deve ser persistido no banco com um campo `locked_at` e timeout de expiração.

---

9. Dashboard Operacional

9.1 Métricas Realmente Úteis

O dashboard deve responder perguntas operacionais concretas, não apenas exibir números:

Métrica	Por que é útil	Query base
Conversas por status	Visão do funil de vendas	`GROUP BY status` em `classifications`
Leads quentes sem resposta há +48h	Prioridade de ação imediata	`last_message_at < now - 48h AND status = 'lead_quente'`
Taxa de conversão por período	Medir eficácia da equipe	`status = 'cliente_ativo'` / total no período

Sentimento médio por semana	Detectar deterioração no atendimento	`AVG(sentiment_score)` por semana
Conversas reclassificadas manualmente	Qualidade das regras automáticas	`classified_by = 'manual'` por período
Tempo médio de resposta	SLA de atendimento	Diferença entre timestamps de mensagens alternadas

9.2 Filtros Essenciais

O dashboard deve oferecer filtros compostos que reflitam o fluxo de trabalho real:

- **Status:** lead_frio, lead_quente, cliente_ativo, suporte, encerrado, indefinido
- **Prioridade:** alta (1), média (2), baixa (3)
- **Intenção:** compra, suporte, dúvida, reclamação
- **Período:** hoje, últimos 7 dias, últimos 30 dias, personalizado
- **Origem da classificação:** automática (regras), IA, manual
- **Sem classificação:** conversas que nunca foram processadas

9.3 Ações Operacionais

O dashboard não é apenas leitura — deve permitir ações que fecham o ciclo:

Plain Text

```
GET  /api/conversations          → lista com filtros e paginação
GET  /api/conversations/:id      → detalhes + mensagens + histórico de classifi
PATCH /api/conversations/:id/classify → override manual de classificação
PATCH /api/conversations/:id/archive  → arquivar conversa
GET  /api/stats/summary          → KPIs consolidados
GET  /api/sync-runs              → histórico de execuções do scheduler
POST /api/sync/trigger           → disparar sync manual
```

O endpoint `PATCH /classify` deve salvar com `classified_by: 'manual'` e a cláusula `WHERE classified_by != 'manual'` no UPSERT garante que o scheduler automático não sobrescreva o override.

---

10. Boas Práticas e Riscos

10.1 Limites do Uso de Biblioteca Não Oficial

A decisão de usar Baileys deve ser consciente e documentada. Os limites práticos são:

- **Não use para envio de mensagens em massa.** O risco de ban é exponencialmente maior. O sistema descrito aqui é somente-leitura, o que é significativamente mais seguro.
- **Mantenha uma única instância por número.** Múltiplas conexões simultâneas para o mesmo número são detectadas e podem resultar em logout forçado.
- **Monitore a versão do Baileys.** Assine as releases no GitHub e teste atualizações em ambiente de desenvolvimento antes de aplicar em produção.
- **Tenha um plano B.** Se o número for banido, o sistema perde acesso imediato. Documente o processo de migração para a API oficial do WhatsApp Business.

10.2 Cuidados com Escala

Para volumes maiores (acima de 10.000 conversas ou 1 milhão de mensagens), algumas adaptações são necessárias:

- **Migrar de SQLite para PostgreSQL:** SQLite tem limitações de concorrência de escrita. Com múltiplos workers ou alta frequência de inserção, o WAL mode do SQLite pode se tornar um gargalo.
- **Processar classificações em workers separados:** Usar `worker_threads` do Node.js para não bloquear o event loop durante chamadas síncronas ao banco.
- **Implementar rate limiting para a API LLM:** Adicionar uma fila com controle de concorrência (ex: `p-limit`) para não ultrapassar os limites de requisições por minuto da API.
- **Particionar mensagens antigas:** Mover mensagens com mais de 6 meses para uma tabela `messages_archive` para manter a tabela principal pequena e rápida.

10.3 Estrutura de Código para Crescer sem Virar Bagunça

A regra fundamental é: **cada módulo depende apenas dos módulos abaixo dele na hierarquia**, nunca dos acima.

Plain Text

```
dashboard/     ← depende de: banco/
scheduler/     ← depende de: classificação/, banco/
classificação/ ← depende de: banco/
ingestão/      ← depende de: banco/
```

```
whatsapp/      ← depende de: (nenhum módulo interno)
banco/         ← depende de: (nenhum módulo interno)
```

Essa hierarquia garante que você pode testar `classificação/` sem precisar de uma conexão Baileys ativa. E pode trocar `whatsapp/` por outra fonte (API oficial, arquivo de exportação) sem tocar em nenhuma outra camada.

10.4 Desacoplamento da Origem das Mensagens

O desacoplamento é feito pelo **barramento de eventos interno** (EventEmitter). A camada `whatsapp/` emite eventos com um schema fixo:

TypeScript

```
// shared/events.ts — contrato público do barramento
interface MessageReceivedEvent {
  id: string;
  chatId: string;
  fromMe: boolean;
  timestamp: number;
  text: string | null;
  messageType: string;
  rawPayload: string;
}
```

Quando você quiser migrar para a API oficial do WhatsApp Business, basta criar um novo módulo `whatsapp-official/` que emite os mesmos eventos com o mesmo schema. Todo o resto do sistema continua funcionando sem modificações.

Conclusão

O sistema descrito aqui é deliberadamente conservador em complexidade: um único processo Node.js, SQLite local, sem filas externas, sem microserviços. Essa simplicidade é uma escolha arquitetural, não uma limitação — ela reduz a superfície de falha, facilita debugging e permite que um desenvolvedor solo entenda e mantenha o sistema inteiro.

O caminho de evolução natural é: (1) adicionar mais regras ao motor local conforme padrões emergem, (2) refinar os prompts LLM com base nas classificações manuais, (3) migrar para PostgreSQL quando o volume exigir, e (4) substituir Baileys pela API oficial quando o negócio justificar o custo.

A arquitetura está preparada para cada um desses passos sem reescrita.